AnalystBot 작업 지시 — 게이트2 오류 영구화 + DeepSeek 구현

Oct 4, 2026 · @jung youngjin

0. 지위·작업 순서

이 문서는 v5 본문과 v5-A 부록 아래의 **실행 지시서**다. 두 문서의 조항을 바꾸지 않고, 10-04 게이트2 세션에서 드러난 결함을 영구 규칙으로 만들고 v5-A(DeepSeek)를 코드로 구현하는 작업 순서를 정한다. `docs/fable/inbox/2026-10-04_workorder_gate2_deepseek.md` 로 넣는다.

작업 순서 — §1(영구화) → §2(규율) → §3(게이트2 완료 보고) → §4(DeepSeek 골격, 스위치 off) → §5(측정·켜기). §1~§3이 끝나기 전에 §4를 시작하지 않는다 — 게이트2가 닫혀야 기준선이 생기고, 기준선이 있어야 DeepSeek 효과를 잴 수 있다.

이 세션에서 이미 고친 것(유지·계약 확인만) — home_id 탑재, done 구조화 인자, ```json 블록 추출, team 열쇠, 날짜 정규식(달+연도), 캐시 단가 분리, BM25 본문 좁히기. 이것들은 §1에서 번호를 받아 계약으로 못 박는다 — 임시 수정으로 남으면 다음 세션·다음 모델에서 같은 계단을 또 밟는다.

1. 게이트2 발견 사항 영구화 — v5 §3-8 "모델 인터페이스" 신설

10-04 세션에서 실측으로 알아낸 Solar의 버릇은 문서에 없던 지식이다. v5 §3에 **§3-8 모델 인터페이스**로 적고, 각 항목을 `tests/contracts/` 의 계약 M47~M52로 만든다. 이미 코드에 들어간 것은 계약·돌연변이가 실제로 물리는지만 확인한다.

번호	규칙	근거(세션 실측)	돌연변이(빨개져야 함)
M47	`done` 은 구조화 인자(JSON 스키마)로 선언하고, 모델이 산문 속 ```json done({…}) 꼴로 내도 루프가 블록을 추출해 수용한다. 추출 실패 2회면 그 경기 중단 사유 `done_parse_fail`	48턴 무한 → 인자 구조화 후 턴 16 done 성공, 산문 JSON 혼합 실측	블록 추출기 끄기
M48	에이전트 출력의 모든 team 칸은 열쇠(`team:NNNNNN`)만. 이름이면 그 항목 폐기 `name_not_key`	done의 team을 이름으로 채운 실측	이름 수용하게 바꾸기
M49	예산 계산은 캐시/비캐시 입력을 분리 단가로 센다. 원장 매 줄에 in·cache·out 토큰과 USD 필수 — None이면 기록 실패	입력 179k 중 캐시 121k, 예산이 제값으로 세어 조기 중단	캐시를 제값 단가로

번호	규칙	근거(세션 실측)	돌연변이(빨개져야 함)
M50	에이전트가 만든 카드 줄은 ✅·이모지·기호로 시작하지 않는다 + 카드 파이프라인은 접두 기호로 줄을 거르지 않는다(둘 다 계약)	에이전트 줄이 ✅로 시작해 카드에서 걸러짐	접두 ✅ 넣기
M51	`AgentInput.match`에 home_id·away_id 필수(§3-1). 없으면 에이전트 호출 전에 실패	game에 home_id 미탑재로 입구 막힘	칸 빼기
M52	검색어 짐작의 날짜 인식은 달+일·달+연도(October 2026)·ISO 전부 포함	'October 2026'이 날짜로 안 세져 검색어가 2요소로 거부됨	달+연도 패턴 제거

추가 한 줄(계약 아님, §3-3 프롬프트에) — 매 8턴마다 시스템이 "지금까지 모은 변수로 done을 부를 수 있다. 카탈로그에서 안 찾은 유형: …"를 리마인드로 넣는다. 48턴 배회의 재발 방지이고, 턴 상한(32)은 그대로 둔다.

2. 작업 규율 추가 — CLAUDE.md §6과 v5 §11-1에 세 줄

이번 세션의 시간 낭비는 봇 결함이 아니라 작업 습관에서 났다. 세 줄을 규칙으로 추가한다.

1. **치환 후 확인** — `str_replace`·`sed`로 코드를 바꾼 뒤에는 바뀐 줄을 `grep -n`이나 `git diff`로 **출력해 보이고 나서** 실행한다. 이번 세션에서 치환이 두 번 빗나갔고(주석만 붙음, import 순서 달라 안 걸림) 둘 다 "고쳤다고 믿고 돌린" 재실행을 낭비했다.
2. **실행 전 WIP 커밋** — 봇·에이전트를 실제로 돌리기 전, 미커밋 변경이 있으면 `wip:` 커밋을 먼저 한다. `git stash`는 쓰지 않는다(이번 세션에서 stash가 작업을 치워 복구 커맨드를 썼다). WIP 커밋은 게이트 통과 커밋에서 squash한다.
3. **프로브 금지(기존 지적의 규칙화)** — 봇 동작 확인은 스크래치패드 스크립트가 아니라 봇의 실제 핸들러(`cmd_analyze` 등)로 한다. 프로브는 인자 파싱·SLATE_DAYS 같은 실제 경로를 건너뛰어 거짓 통과를 만든다. 임시 스크립트가 필요하면 실제 핸들러를 import해서 부른다.

3. 게이트2 완료 조건과 보고

게이트2의 정의는 v5 §10-4-2다: **정답지 5경기에서 깔때기** **`verified_vars ≥ 6`**. 지금은 MLB 한 경기로 루프를 세웠을 뿐이다. 완료하려면:

1. 수정된 예산(ao8)의 재실행 결과를 보고한다 — 턴 수, done 턴, 검색어 통과/거부, verified_vars, 사유별 폐기 수, in·cache·out 토큰과 USD.
2. 정답지에서 종목이 다른 5경기를 고른다(MLB 1·NPB 1·KBO 1·네이션스리그 2 권장 — 언어·어댑터·대표팀 경로가 전부 밟힌다). 각각 `cmd_analyze`로 돌린다.
3. 경기마다 깔때기 한 줄과 변수 목록(유형·방향·hardness·인용 유무)을 보고한다.

4. 5경기 중 4경기 이상 `verified_vars ≥ 6`이고 M47~M52 돌연변이가 전부 빨개지면 게이트2 통과 — `stage_gate.yaml`에 서명하고 커밋 해시를 적는다.
5. 미달 경기는 깔때기에서 어느 단계가 0으로 떨어졌는지 사유 코드로 보이고, 수정은 §11-1(증상 수정 금지 — 같은 꼴이면 계약부터)대로.

보고에 반드시 넣을 것 — 테스트 통과/skip/돌연변이 수(전·후), 비용 합계, 커밋 해시, `questions/`에 남긴 갈림길. 로그 원문 없이 "됐다"는 보고는 받지 않는다(v5 §11-2 그대로).

4. DeepSeek 구현 작업 목록 — v5-A를 코드로, 스위치 off

게이트2가 닫힌 뒤 시작한다. 전부 **`deepseek.enabled: false`** 상태로 만들고, 켜는 것은 §5 측정 뒤 사용자 승인이다. 구현 중에도 Solar 단독 경로가 깨지지 않아야 한다(M45).

1. **설정** — `config/llm.yaml`에 `deepseek:` 칸(모델 ID·base_url·입력/캐시/출력 단가·확인 날짜·enabled:false). 단가는 공식 api-docs·pricing 페이지를 직접 읽어 적는다(v5-A §2). `.env.example`에 `DEEPSEEK_API_KEY` 이름 추가.
2. **클라이언트** — `app/llm/deepseek.py`: OpenAI 호환, **tools 없음**(M42), JSON 출력 모드, 타임아웃 120초·재시도 2회, usage를 `provider:deepseek`로 원장 기록(M49와 같은 칸). 장애 시 예외를 삼키고 `deepseek_unavailable` 반환(M45).
3. **단계 3** — `app/hypo.py`: `HypoInput` 조립(검증 변수·구조 사실·보드 줄· `required_types`는 코드가 v5 §6-2 조건으로 선계산·Solar 초안은 참고로), 호출, 코드 처리(driver_vars가 검증 ID 밖이면 `unknown_var` 폐기, mechanism에 입력 밖 고유명사면 `new_fact` 폐기, outcomes 범위 검사, 남은 세트에 §6-2 자동 보강 — 보강 발생 시 `deepseek_missed_required` 기록).
4. **단계 4** — `app/redteam.py`: 1차 생존 0.8 이상 마켓이 있을 때만 호출. 마켓별 kill_range → covered/proposed/impossible. proposal은 단계 3과 같은 검사 뒤 `added_by:redteam`으로 세트에 추가. **추가만 가능** — 삭제·약화 금지, 추가 뒤 재판정에서 어떤 마켓의 생존율도 올라가면 카드 금지(M44). 필수형 조건이 걸리는데 impossible이면 템플릿 투입 `redteam_overruled`(M46).
5. **계약** — M41~M46을 `tests/contracts/`에, v5-A §5의 돌연변이 그대로. **정답지 재현 계약**: 남아공 5-0·콜롬비아–파라과이·크로아티아 봇 카드 세 경기를 고정 입력으로, 실제 결과를 만든 갈래(대량 득점·파라과이 승·난타전)가 세트에 들어가야 통과 — 이 테스트가 빨간 동안 enabled를 켜지 않는다.
6. **사유 코드 추가** — `unknown_var, new_fact, redteam_skipped, redteam_overruled, deepseek_unavailable, deepseek_missed_required` 6종을 v5 §8-6 enum에.
7. **판정 순서 변경** — v5 §6을 "보강 → 1차 대입 → 반증 → 재대입 → 상한 → pick"으로(v5-A §7 표). enabled=false면 반증 단계를 건너뛰고 로그에 `redteam:off` 한 줄.

5. 측정·켜는 순서

v5-A §6 그대로 하되, 게이트 흐름에 끼우는 자리를 못 박는다.

1. 게이트3(검증층 가짜 인용 10건 전부 폐기) → 게이트4(재현율 70%/50%) → 게이트5(카드 비교)를 **Solar 단독**으로 통과한다. DeepSeek 코드는 있지만 꺼져 있다.
2. 게이트5 결과를 `metrics/baseline_solar.json`으로 고정한다 — 필수형 누락률(`added_by:code` 비율)·결과 갈래 포함률·거짓 확신 수·오답 재현 수·등급 마켓 수·경기당 비용.
3. `enabled: stage3`로 단계 3만 켜고 같은 정답지 20경기를 재실행 → `metrics/stage3.json`.
4. `enabled: stage3+4`로 반증까지 켜고 재실행 → `metrics/stage34.json`.
5. `metrics/compare.py`가 세 파일을 표로 비교해 보고한다. 켜는 기준(v5-A §6): 결과 갈래 포함률 상승 + 오답 재현 수 감소 또는 동일 + 등급 마켓 수가 기준선의 70% 이상 → 단계 3 운영 켬. 거짓 확신 수 감소 → 단계 4도 켬. 기준 미달이면 `questions/`에 수치와 함께 적고 멈춘다 — 사용자가 정한다.
6. 운영 뒤 원장 30건마다 같은 지표를 다시 뽑아 보고하고, DeepSeek 비용이 하루 예산 절반을 넘으면 단계 4를 생존 0.85+ 마켓으로 좁힌다.

6. 전체 실행 순서 요약과 금지 사항

순서 — ① §1 M47~M52 계약화(+§3-3 리마인드) → ② §2 규율 세 줄을 CLAUDE.md에 → ③ ao8 결과 보고 → ④ 정답지 5경기 게이트2 완료·서명 → ⑤ §4 DeepSeek 골격(enabled:false) + M41~M46 + 정답지 재현 계약 → ⑥ 게이트3~5 Solar 단독 → ⑦ 기준선 고정 → ⑧ stage3 측정 → ⑨ stage3+4 측정 → ⑩ 비교 보고, 켜기는 사용자 결정.

금지 — §1~§3 전에 §4 시작 금지 / enabled 기본값을 true로 커밋 금지 / 정답지 재현 계약(남아공·콜롬비아·크로아티아)이 빨간 채로 DeepSeek 켬 금지 / DeepSeek 호출에 tools 전달 금지 / 반증 단계의 시나리오 삭제·생존율 상승 금지 / `git stash` 금지 / 치환 후 diff 확인 없이 실행 금지 / 스크래치패드 프로브 금지.

각 단계 커밋 메시지에 이 문서의 §번호와 M번호를 적는다. 갈림길은 `questions/`에 적고 멈춘다 — 혼자 고르지 않는다.